WEEK 5 – PAYMENT TRANSACTION MANAGEMENT SYSTEM

DEPARTMENT OF COMPUTER APPLICATIONS

DBMS PROJECT DOCUMENTATION

Title: Payment Transaction Management System

Done by: S. Veeramanikandan

Department: Bachelor of Computer Applications (BCA)

Class: II-BCA – "A"

OBJECTIVE

To design and implement a Payment Transaction Management System using MySQL. The system stores payment details, maintains payment status, connects payments with orders, and generates transaction and revenue reports.

1. PAYMENT TABLE DESIGN

The Payment table stores the payment information associated with customer orders. It includes the payment mode, transaction amount, payment date, and payment status.

Table Structure

Column Name	Data Type	Description
Payment_ID	INT	Primary key with auto-increment
Order_ID	INT	Foreign key referencing Orders
Payment_Date	DATE	Date of payment
Payment_Mode	VARCHAR(30)	Method used for payment
Payment_Status	VARCHAR(20)	Pending, Success, or Failed
Transaction_Amount	DECIMAL(10,2)	Amount of the transaction

2. UPDATED ER DIAGRAM – ORDER AND PAYMENT RELATIONSHIP

Relationship: Orders → Payment

The Orders table contains the order details, while the Payment table contains the corresponding payment information.

- `Orders.Order_ID` is the primary key of the Orders table.
- `Payment.Payment_ID` is the primary key of the Payment table.
- `Payment.Order_ID` is a foreign key referencing `Orders.Order_ID`.
- The UNIQUE constraint on `Payment.Order_ID` allows at most one payment record per order.

ER Diagram

CUSTOMER

Customer_ID (Primary Key)

Customer details

↓ One customer can place multiple orders

ORDERS

Order_ID (Primary Key)

Customer_ID (Foreign Key)

Order_Date

Total_Amount

Order_Status

↓ One order can have zero or one payment record

PAYMENT

Payment_ID (Primary Key)

Order_ID (Foreign Key, UNIQUE)

Payment_Date

Payment_Mode

Payment_Status

Transaction_Amount

3. SAMPLE CUSTOMER, ORDER AND PAYMENT DATA

Customer Data

The Customer table contains customer records identified by Customer_ID. The customer names and email addresses can be displayed using:

```
SELECT * FROM Customer;
```

Order Data

Order_ID	Customer_ID	Total Amount (₹)	Order Status
1	1	1,800.00	Pending
2	2	1,100.00	Pending
3	3	1,900.00	Pending
4	4	1,500.00	Pending
5	5	800.00	Pending

Payment Data

Payment_ID	Order_ID	Payment Mode	Payment Status	Amount (₹)
6	1	UPI	Success	1,800.00
7	2	Credit Card	Failed	1,100.00
8	3	Debit Card	Success	1,900.00
9	4	Net Banking	Success	1,500.00
10	5	Cash on Delivery	Pending	800.00

The Payment_ID values start at 6 because of previous auto-increment activity in MySQL. The current payment records are valid.

4. PAYMENT TRANSACTION AND REVENUE ANALYSIS REPORTS

A. Payment Status Summary

Payment Status	Number of Payments	Amount (₹)
Success	3	5,200.00
Failed	1	1,100.00
Pending	1	800.00

Total	**5**	**7,100.00**

B. Display Successful Payments

```
SELECT *
FROM Payment
WHERE Payment_Status = 'Success';
```

C. Display Failed Payments

```
SELECT *
FROM Payment
WHERE Payment_Status = 'Failed';
```

D. Display Pending Payments

```
SELECT *
FROM Payment
WHERE Payment_Status = 'Pending';
```

E. Count Payments by Status

```
SELECT Payment_Status,
       COUNT(*) AS Total_Payments
FROM Payment
GROUP BY Payment_Status;
```

F. Calculate Successful Transaction Amount

```
SELECT SUM(Transaction_Amount) AS Successful_Amount
FROM Payment
WHERE Payment_Status = 'Success';
```

Result: ₹5,200.00

G. Count Transactions by Payment Mode

```
SELECT Payment_Mode,
       COUNT(*) AS Total_Transactions
FROM Payment
GROUP BY Payment_Mode;
```

H. Display Order and Payment Details

```
SELECT o.Order_ID,
       o.Customer_ID,
       o.Order_Date,
       o.Total_Amount,
       p.Payment_Mode,
       p.Payment_Status,
       p.Transaction_Amount
FROM Orders o
JOIN Payment p
ON o.Order_ID = p.Order_ID;
```

I. Update Payment Status

The following query updates the Debit Card payment for Order_ID 3 from Pending to Success.

```
UPDATE Payment
SET Payment_Status = 'Success'
WHERE Order_ID = 3;
```

To verify the updated records:

```
SELECT * FROM Payment;
```

Analysis

The Payment table contains five payment records. Three transactions are successful, one transaction has failed, and one transaction is pending. The total amount associated with successful transactions is ₹5,200.00. The Debit Card transaction for Order_ID 3 has been updated successfully.

The successful transaction amount represents payments marked Success; it should not automatically be treated as total business revenue without considering the application's accounting rules.

5. QUERY OUTPUT SCREENSHOTS

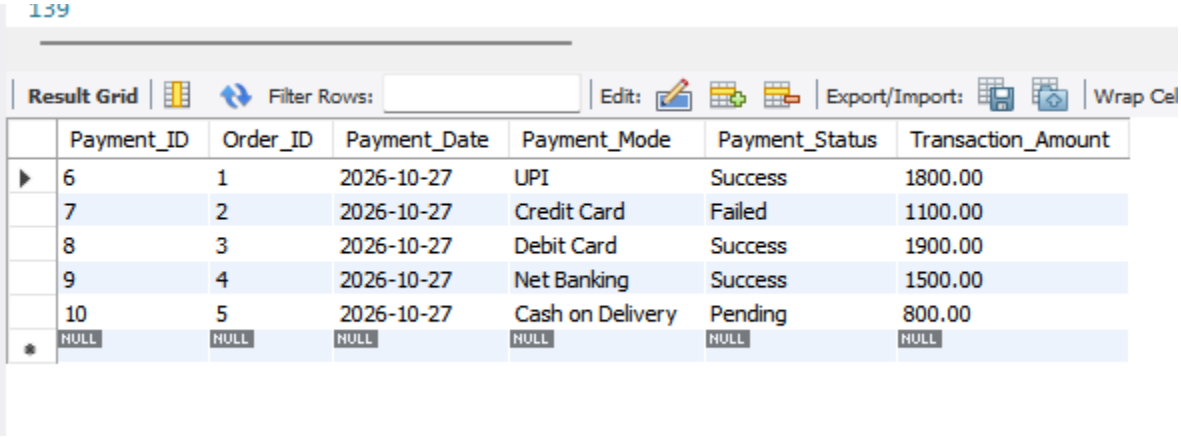
Screenshot 1: Payment Table Output

Insert the supplied MySQL Workbench screenshot here. It shows the Payment table after the Debit Card payment for Order_ID 3 was updated to Success.

Caption: Figure 1 – Output of the Payment Transaction Management System in MySQL Workbench.

The screenshot displays the Payment_ID, Order_ID, Payment_Date, Payment_Mode, Payment_Status, and Transaction_Amount columns.

Screenshot of Result:


139

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cel

Payment_ID	Order_ID	Payment_Date	Payment_Mode	Payment_Status	Transaction_Amount
6	1	2026-10-27	UPI	Success	1800.00
7	2	2026-10-27	Credit Card	Failed	1100.00
8	3	2026-10-27	Debit Card	Success	1900.00
9	4	2026-10-27	Net Banking	Success	1500.00
10	5	2026-10-27	Cash on Delivery	Pending	800.00
NULL	NULL	NULL	NULL	NULL	NULL

CONCLUSION

The Payment Transaction Management System was implemented successfully using MySQL. The Payment table stores payment details and maintains the relationship between orders and payments using a foreign key. SQL queries were used to display payment records, identify successful, failed, and pending transactions, update payment status, and calculate the successful transaction amount.

This system demonstrates the use of primary keys, foreign keys, UNIQUE constraints, CHECK constraints, aggregate functions, joins, and UPDATE statements in database management.